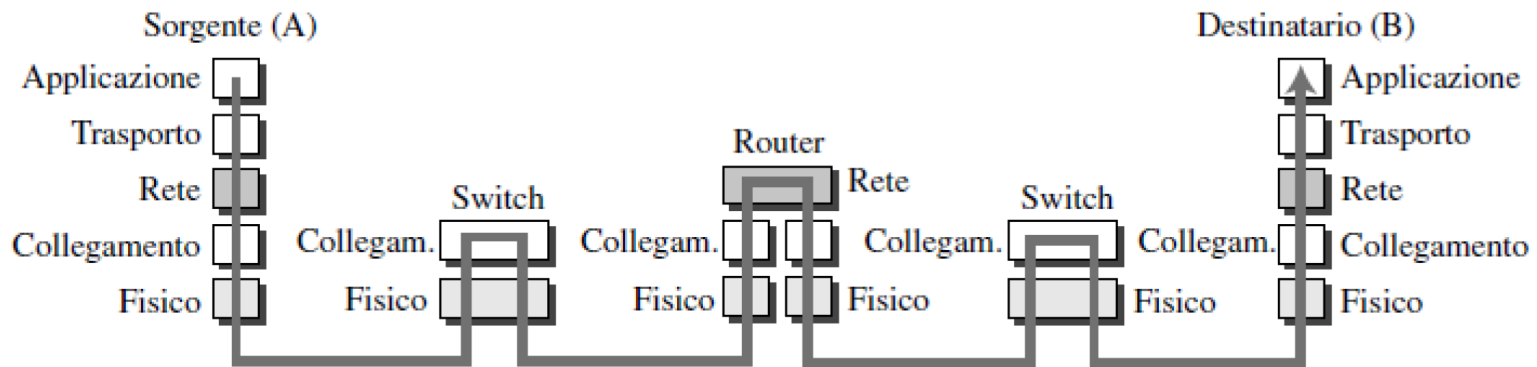


Introduzione al livello di trasporto

Livello di trasporto



- ▶ Il compito del livello trasporto è di **fornire un trasporto efficace dall'host di origine a quello di destinazione**, indipendentemente dalla rete utilizzata.
- ▶ Questo è il livello in cui si gestisce per la prima volta (dal basso verso l'alto) una **conversazione diretta**, cioè senza intermediari, fra sorgente e destinazione.



Attenzione: per semplificare il diagramma in figura sono stati omessi i livelli sessione e presentazione.

Livello di trasporto



- ▶ Il **livello di trasporto** del **mittente** riceve i dati dai livelli superiori, li compone in "segmenti" e vi aggiunge un'intestazione passando al livello rete che, a sua volta, li imbusta e li inoltra al livello di collegamento che, tramite il livello fisico, li invia sul canale
- ▶ Il sistema del **destinatario** opera al contrario: toglie le intestazioni dei pacchetti e ricostruisce il messaggio originale che consegna all'applicazione destinataria
- ▶ Il **livello di trasporto** svolge un compito di collegamento tra il sottostante livello di rete e il livello applicativo, instaurando un **collegamento logico** tra le applicazioni residenti su host remoti, mettendo in comunicazione i processi software.
- ▶ Principali caratteristiche:
 - ▶ **indipendenza dal tipo di rete utilizzata**
 - ▶ **affidabilità** e **qualità** della trasmissione
 - ▶ **multiplexing e demultiplexing** (pacchetti relativi ad applicazioni diverse di essere ricevuti da un sistema remoto in modo corretto)



Livello di trasporto



I servizi offerti dallo strato di trasporto sono sostanzialmente di due tipi:

- ▶ **trasporto affidabile**: garantisce che i messaggi arrivino a destinazione nel corretto ordine di invio
- ▶ **trasporto non affidabile**: garantisce solo funzionalità di indirizzamento; nessuna garanzia di consegna



Livello di trasporto



Servizio affidabile e servizio non affidabile

Un servizio è **affidabile** quando non c'è perdita di dati durante la trasmissione:

- ▶ tutti i dati spediti vengono consegnati al destinatario, oppure viene segnalato un errore;
- ▶ il ricevente invia un **acknowledgment** (ACK) di conferma al mittente.

Un servizio si dice invece **non affidabile** se non offre alcuna certezza che i dati inviati siano effettivamente ricevuti.



Livello di trasporto



Lo strato di trasporto della rete Internet mette a disposizione delle applicazioni attive in ciascun host, due distinti protocolli di trasporto:

- ▶ **servizi affidabili orientati alla connessione**, detti di tipo stream offerti dal **TCP** (*Transmission Control Protocol*);
- ▶ **servizi senza connessione** detti di tipo datagramm offerti dall'**UDP** (*User Datagram Protocol*).



Livello di trasporto

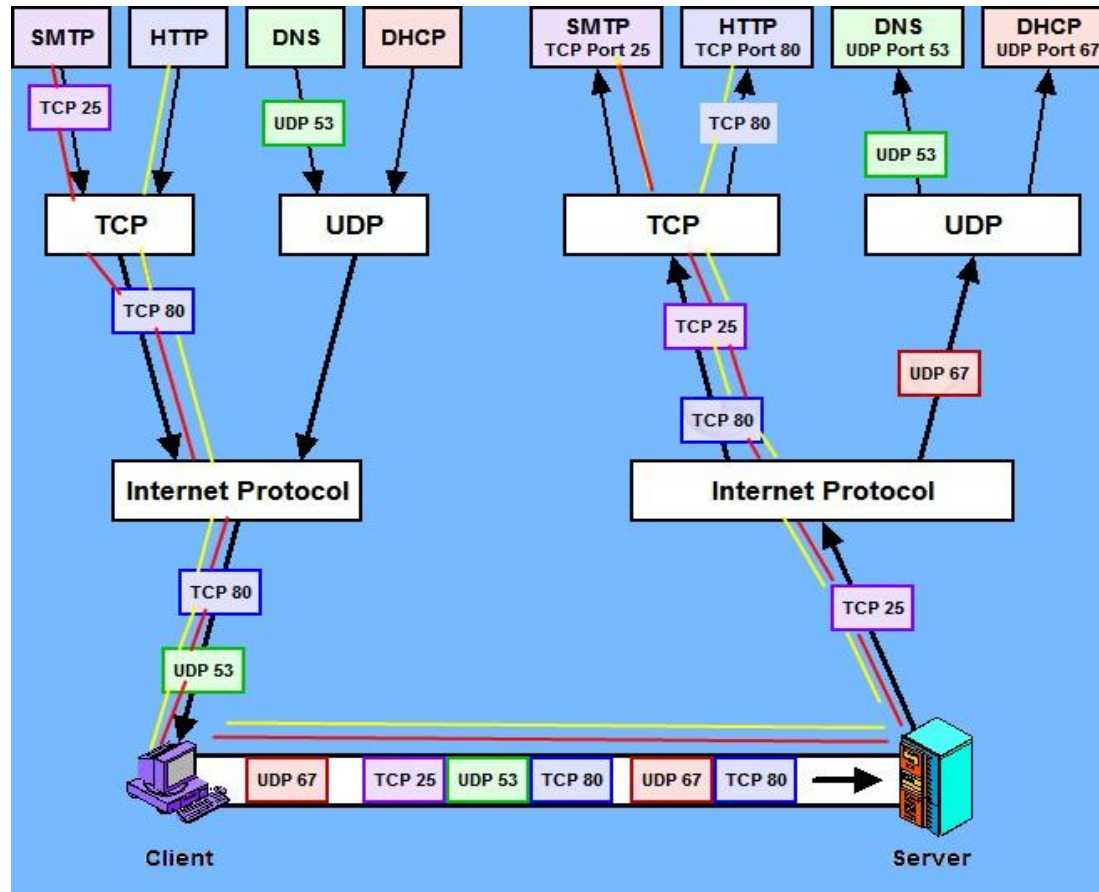


Il protocollo **TCP** offre un **trasporto affidabile** in quanto consente il controllo dell'integrità dell'informazione contenuta nei pacchetti e il controllo sull'effettiva consegna del messaggio (uso messaggi di acknowledge),

Il protocollo **UDP** è senza connessione (connectionless) e offre un **trasporto non affidabile**.



TCP e UDP Multiplexing e Demultiplexing



- ▶ Multiplexing e Demultiplexing sono due funzionalità dei protocollo TCP e UDP che permettono ad un messaggio di essere direzionato in maniera corretta, sia nel percorso di andata sia nel percorso di ritorno del Modello OSI. Più messaggi viaggiano insieme sullo stesso 'canale' e
- ▶ raggiungono la destinazione.

UDP (User Datagram Protocol)



- ▶ È un protocollo di trasporto semplice che offre il minimo valore aggiunto possibile (multiplexing) rispetto a IP con l'obiettivo di richiedere il minimo possibile di risorse aggiuntive
 - ▶ È definito nella **RFC768** (<https://tools.ietf.org/html/rfc768>)
 - ▶ UDP si usa con protocolli applicativi che hanno bisogno dei vantaggi di TCP
 - ▶ Fornisce un servizio non affidabile:
 - i pacchetti possono andar persi o essere consegnati non in ordine;
 - gli utenti scambiano datagram (non flussi);
 - possibili trasferimenti in parallelo in entrambe le direzioni (full duplex).
 - ▶ Le PDU di livello trasporto (TPDU), quando si utilizza il protocollo UDP, si chiamano **datagrammi**.
-



Recapito UDP



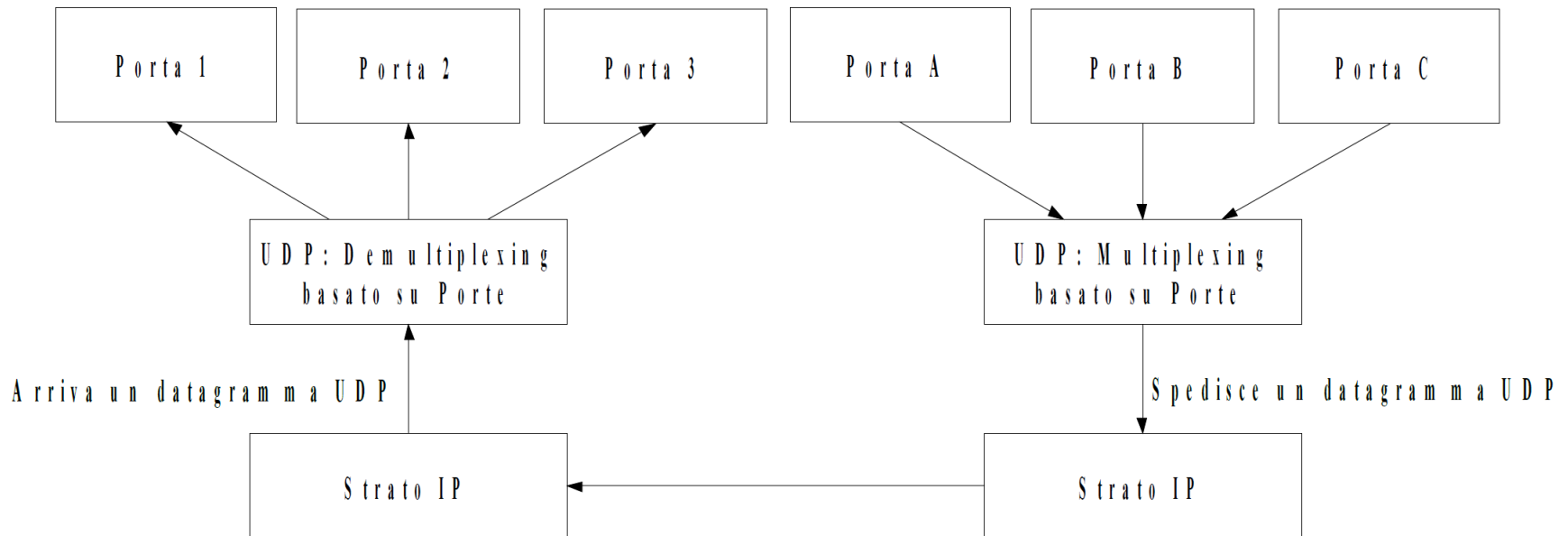
- ▶ UDP fornisce un servizio di recapito NON affidabile:
 - ▶ **senza connessione** (**connectionless**) ;
 - ▶ **senza buffering**: accetta i dati e li trasmette immediatamente (nessun servizio di buffering prima della trasmissione).
- ▶ Per certe applicazioni la mancanza di affidabilità non costituisce un problema.

Esempio - Trasmissione di flussi audio

Se si perde un pacchetto ogni tanto la comprensione del flusso da parte del destinatario NON ne risulta compromessa.



Recapito UDP: Multiplexing e Demultiplexing



- ▶ UDP supporta *multiplexing* e *demultiplexing* basato sulle porte
- ▶ In UDP la porta sorgente è opzionale (vale zero quando non è usata)

Formato del Datagram



0	4	8	16	24	31
Porta sorgente			Porta di destinazione		
Lunghezza del messaggio			Checksum		
Dati					

- ▶ **Porta di destinazione:** identifica il processo di destinazione.
- ▶ **Porta sorgente (opzionale):** identifica il processo sorgente per le risposte, oppure vale zero.
- ▶ **Lunghezza del messaggio:** lunghezza del datagram in byte, compresi intestazione e dati.
- ▶ **Checksum (opzionale):** checksum di 16 bit su intestazione e dati, oppure zero.



Differenze tra TCP e UDP



- ▶ **La scelta di UDP rispetto a TCP si basa su:**

- ▶ Funzionalità
- ▶ Prestazioni

Prestazioni a confronto

Lo schema di controllo del flusso di TCP basato sulle finestre porta a *bulk transfer* a ondate:

- ▶ I meccanismi per la gestione dell'affidabilità possono ridurre il *throughput*
- ▶ TCP ha un elevato *overhead* per segmento (header più grande rispetto a UDP)



Differenze tra TCP e UDP



▶ **Affidabilità**

- ▶ TCP fornisce trasferimenti affidabili e ordinati
- ▶ UDP fornisce un servizio inaffidabile, l'applicazione deve accettare o considerare:
 - perdite di pacchetti dovute a overflow ed errori;
 - datagram non in ordine.

▶ **Multicast e broadcast**

- ▶ Supportati solo da UDP.
- ▶ Lo schema di controllo degli errori di TCP non si presta al multicast affidabile.



Protocollo TCP



TCP è dunque un protocollo orientato alla connessione, il software di rete che implementa TCP deve assicurare due condizioni fondamentali:

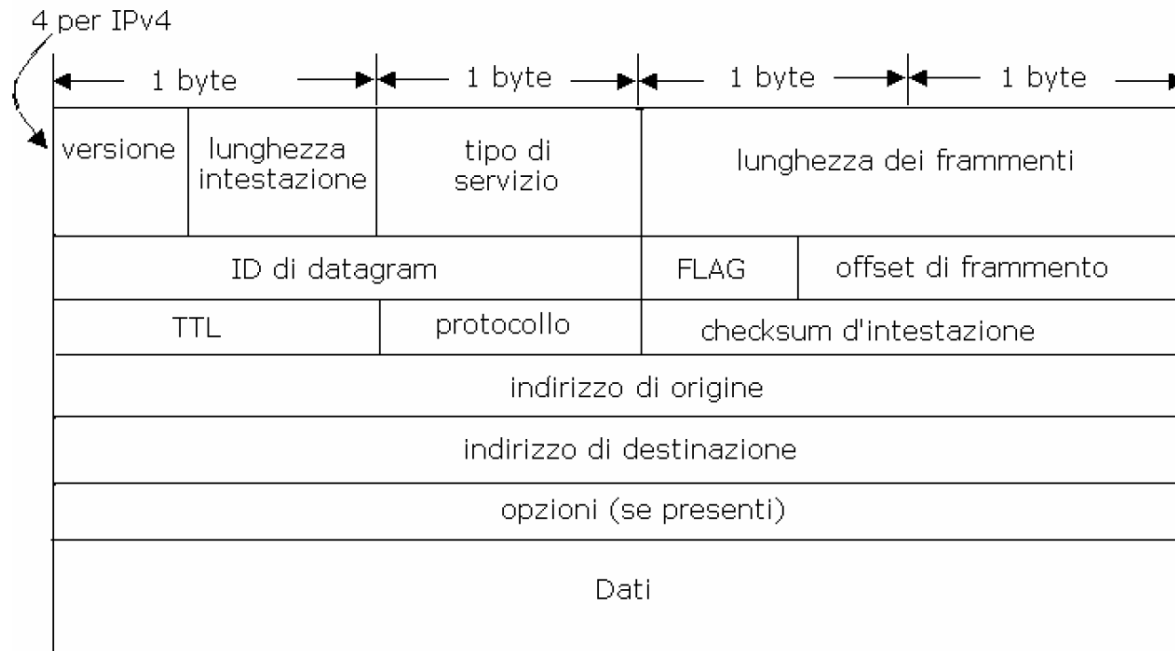
- ▶ certezza che il programma applicativo destinatario sia attivo;
- ▶ garanzia che tutti i pacchetti inviati dal mittente raggiungeranno la loro destinazione.
- ▶ Le PDU di livello trasporto (TPDU) quando si utilizza il protocollo TCP si chiamano **segmenti**.



Protocollo TCP



Per raggiungere gli obiettivi di TCP, l'**intestazione IP** non è sufficiente...



IP header

È necessario aggiungere un'ulteriore intestazione da incapsulare in ogni pacchetto IP da inviare.

Protocollo TCP



TCP header

Porta Sorgente(16)			Porta destinazione(16)		
Numero di Sequenza(32)					
Numero di Acknowledgement(32)					
HLEN(4)	Riservati(6)	Flag(6)	Window(16)		
Checksum(16)			Urgent Pointer(16)		
Opzioni				Padding	
Dati					



Protocollo TCP



Elementi dell'intestazione TCP

▶ Numero di porta sorgente TCP

Campo da 16 bit con cui viene identificato il processo mittente che invia i dati. Lato client la porta sorgente viene scelta dal sistema operativo tra le porte 'alte' (IANA suggerisce di usare porte nel range 49152-65535, da RFC devono comunque essere > 1024); lo stesso numero di porta verrà inserito come porta destinazione dal processo destinatario quando invierà la risposta. Lato server: il server può usare come porta sorgente la porta su cui ha ricevuto la richiesta del client (es. 80, 443, ...)

▶ Numero di porta di destinazione TCP

Campo da 16 bit con cui viene identificato il processo destinatario a cui inviare i dati. La porta destinazione è generalmente una porta convenzionalmente associata a una particolare servizio applicativo (es. porta 80 per http, 443 per https, ...).

▶ Numero di sequenza

E' un campo di 32 bit che, con il numero di acknowledgement, permette di mantenere traccia del numero di byte inviati e ricevuti nei segmenti relativi a un flusso dati TCP. TCP non numera i singoli segmenti ma i singoli byte: i dati trasportati sono, infatti, considerati come un unico flusso di byte (stream), numerati a partire dall'apertura di connessione. Il numero di sequenza associato al primo byte del segmento viene scelto casualmente dal sistema operativo. Se, per esempio, vengono inviati 3 messaggi da 2048 byte, ipotizzando che il numero iniziale sia 0, il numero di sequenza del primo segmento sarà 0, quello del secondo 2048, quello del terzo 4096.

▶ Numero di conferma di ricezione (acknowledgement number ack)

E' un campo di 32 bit che identifica il numero del byte successivo all'ultimo ricevuto. Esso indica al trasmittente remoto, quale dovrà essere il successivo numero di byte che si aspetta di ricevere. Se, per esempio, è stato ricevuto un messaggio di 2048 byte accettato come valido, contenente come numero di sequenza il valore 0, nel successivo segmento che il ricevente invierà al mittente inserirà un numero di ack pari a 2048. Ciò indica che sono stati ricevuti correttamente 2048 byte (dal byte 0 al byte 2047) e che si aspettano i successivi a partire dal byte numero 2048.

Protocollo TCP



► Somma di controllo TCP (*checksum*)

E' il campo utilizzato per il controllo degli errori di trasmissione. La modalità di calcolo verrà presentata più avanti.

► Dimensioni della finestra a scorrimento TCP

E' il campo che sfrutta il meccanismo della finestra scorrevole,. Indica il numero di byte ancora liberi che il ricevente è in grado di accettare senza saturare il proprio buffer. Il valore zero indica che il mittente deve fermarsi e non inviare dati fino a successiva comunicazione. Questo meccanismo verrà presentato nel dettaglio più avanti.

► Bit di segnalazione (FLAG)

Sono 8 bit utilizzati per gestire le opzioni di controllo del protocollo. Si impostano a 1 quando devono essere attivati.

Il bit SYN è utilizzato per indicare che si sta aprendo una nuova connessione.

Il bit FIN si utilizza in fase di chiusura connessione.

Il bit ACK viene messo a 1 per indicare che il campo *Acknowledgment* contiene dati che devono essere considerati (viene sempre posto a 1 tranne che nel messaggio di apertura connessione)

Il bit RST viene utilizzato per resettare la connessione in seguito a un errore. Quando si riceve un messaggio con RST=1 significa che la connessione interessata deve essere chiusa immediatamente (chiusura non corretta, a causa di errore)

Il bit PSH si utilizza per indicare al destinatario che i dati devono essere immediatamente consegnati al livello superiore (es. si usa per inviare comandi su un terminale remoto)

Il bit URG indica al destinatario che il segmento trasporta dati urgenti (es. per gestire un interrupt relativo alla pressione, lato mittente, di CTRL+ALT+CANC in un'applicazione utilizzata per il controllo remoto). I dati urgenti saranno inseriti a partire dallo scostamento in byte indicato nel campo *Urgent Pointer*.

I bit CWR e ECE vengono usati rispettivamente per indicare un rallentamento nell'invio dati e per cercare di prevenire la perdita di pacchetti in caso di congestione.



Protocollo TCP



- ▶ **Bit di segnalazione (FLAG)**

Utilizzato insieme al flag URG = 1 per indicare a partire da quale byte i dati inviati sono urgenti.

- ▶ **Options**

E' è un campo facoltativo utilizzato per eventuali opzioni. Per esempio può essere utilizzato per utilizzare il selective repeat, per la ripetizione di uno o più segmenti, oppure go-back-n.

- ▶ **Dati**

Dopo l'intestazione sono inseriti i dati da trasferire, ossia il payload, corrispondente alla PDU proveniente dal livello superiore.



Protocollo TCP



- ▶ Per capire il significato del *numero di sequenza*, bisogna ricordare che i *segmenti TCP* viaggiano in un *ordine sequenziale numerato*, all'interno di pacchetti IP. Il numero di sequenza nell'intestazione TCP *stabilisce l'ordine che la destinazione deve usare per riassemblare i segmenti nell'ordine di partenza*.
- ▶ Quando l'host ricevente ottiene un segmento TCP, risponde al mittente con un piccolo pacchetto di conferma detto **ACK** (*ACKnowledgment*) o *conferma di ricezione*. Il numero di ciascuna conferma di ricezione coincide con il numero di sequenza del pacchetto che è stato ricevuto più uno.



Protocollo TCP



- ▶ **Il mancato ACK viene rilevato dal mittente:** se non riceve una conferma di ricezione per ogni pacchetto che ha trasmesso, trascorso un tempo t di timeout, il mittente rimanda il pacchetto in questione.
- ▶ Nella pratica, per ridurre il numero di conferme (ACK) ed ottimizzare lo scambio dei dati, gli host scambiano anche un numero relativo alla dimensione della finestra (**campo window nell'header**): questo numero indica quanti byte possono essere ricevuti e mantenuti nel buffer prima di inviare una conferma (ack). La finestra viene adattata in base alle condizioni del trasferimento (es. errori rilevati) regolando il flusso TCP.



Protocollo TCP



Three way handshake

Il procedimento per avviare una connessione TCP può essere informalmente descritto come segue:

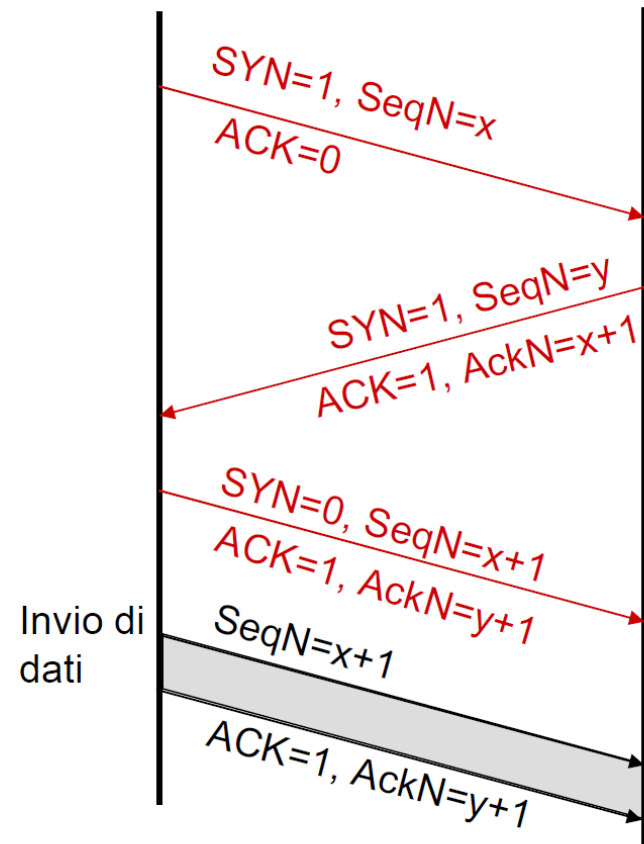
- ▶ *"Iniziamo una connessione, fammi sapere se sei in linea e hai ricevuto questa richiesta". Passo 1.*
- ▶ *"Sì, io ho ricevuto la tua richiesta e sono pronto a stabilire il collegamento ". Passo 2.*
- ▶ *"Va bene, ho ricevuto la tua conferma di ricezione; ecco i primi dati per te". Passo 3 (connessione stabilita).*
- ▶ La sequenza iniziale con cui viene stabilita una connessione è detta *three-way handshaking* o *sincronia trilaterale*



Protocollo TCP



- ▶ L'apertura della connessione non è un processo banale perché la rete può perdere, duplicare o ritardare i pacchetti scambiati
- ▶ TCP usa uno schema detto **Three Way Handshake** che risulta essere molto robusto
 - sincronizzazione di entrambi i numeri di sequenza
- ▶ Il primo pacchetto dati ha numero di sequenza uguale all'ACK precedente
 - ACK non occupa spazio di numerazione



Protocollo TCP



- ▶ Stabilita la connessione, il flusso dei dati è **affidabile, in sequenza e bidirezionale**
- ▶ Il flusso, come anticipato, viene regolato tramite un buffer d'invio e uno di ricezione, mediante delle tecniche che verranno approfondite più avanti.

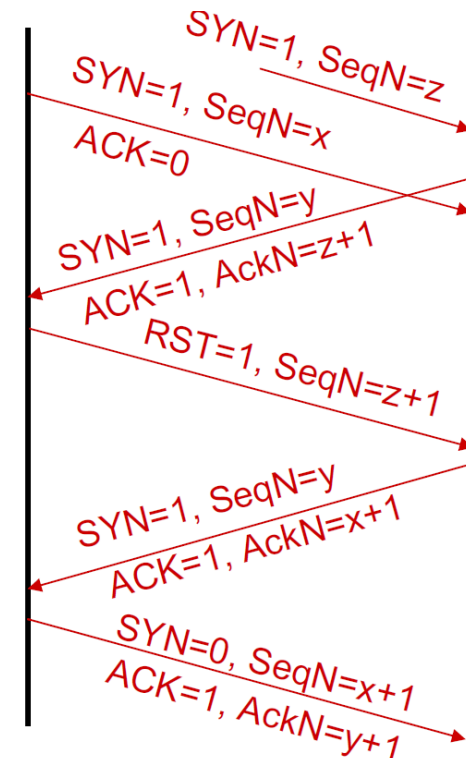
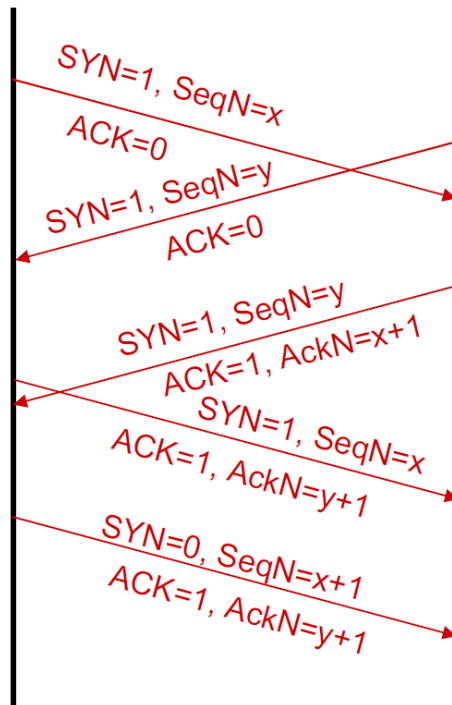


Protocollo TCP (approfondimento)



▶ II **three-way handshake**

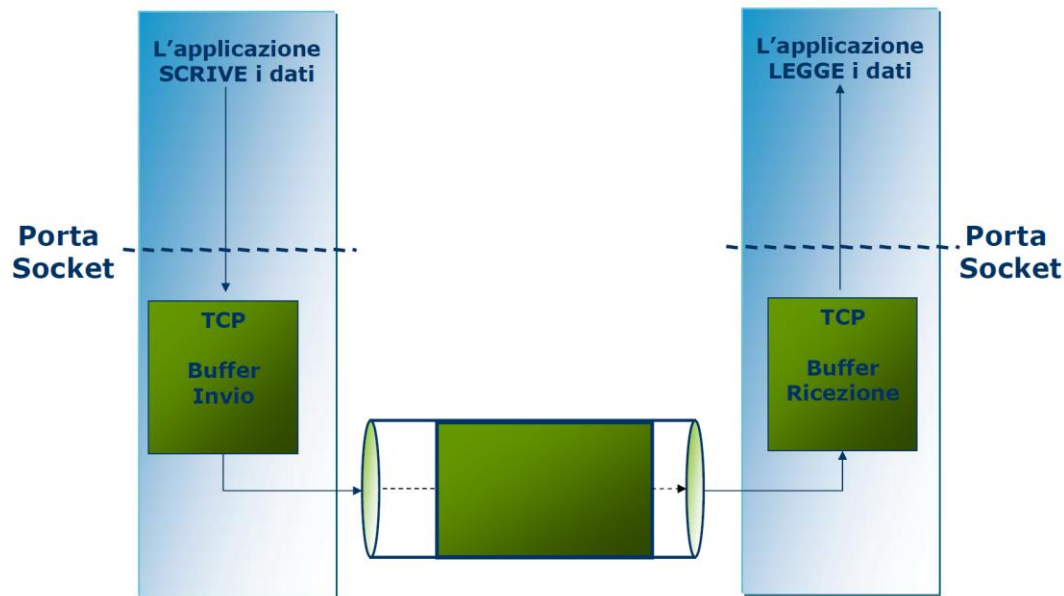
- resiste all'avvio contemporaneo di due connessioni
- ignora pacchetti di apertura ritardatari



Protocollo TCP



- ▶ TCP utilizza il protocollo IP per trasportare i messaggi ma, rispetto a quest'ultimo, distingue tra più destinazioni all'interno di uno stesso host mediante il meccanismo delle porte (**socket**).
- ▶ **Socket**: astrazione utilizzata per interfacciare i due terminali (*endpoint*) in gioco in una connessione tra due computer.



Protocollo TCP

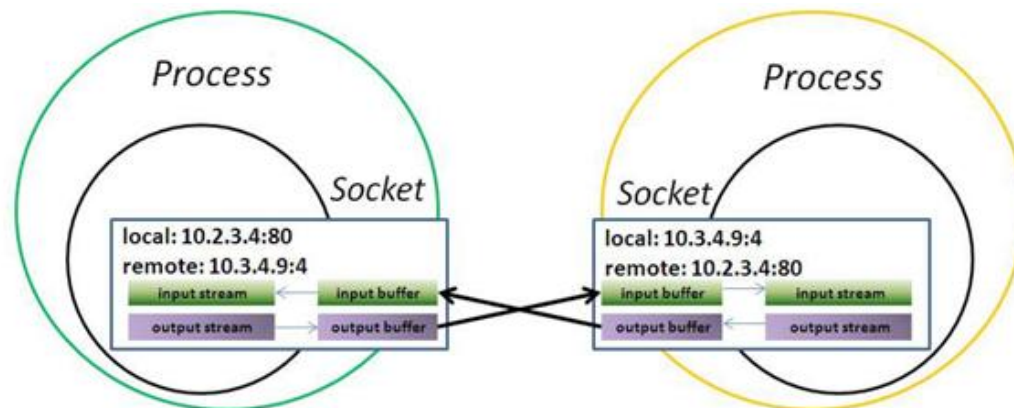


Una connessione TCP viene dunque è identificata univocamente da una quadrupla:

<IP sorgente; porta sorgente; IP destinazione; porta destinazione>

La notazione per esprimere la <IP, porta> è convenzionalmente IP:Porta

TCP è definito in RFC 793, estensioni in RFC 1323



Protocollo TCP



Quando arriva una connessione viene attivato un unico processo (programma **inetd**) che intercetta le richieste di connessione, crea il processo relativo e ridirige la connessione da se stesso al processo appena creato, liberandosi per intercettare future richieste.

Inetd determina a quale programma la connessione è destinata, esegue quel particolare processo e affida a lui il socket.

Le porte con numero inferiore a 1024 sono porte riservate per specifici servizi (well-known ports)



Protocollo TCP



Principali Well-Known Port

Port Number	Transport Protocol	Service Name	RFC
20, 21	TCP	File Transfer Protocol (FTP)	RFC 959
22	TCP and UDP	Secure Shell (SSH)	RFC 4250-4256
23	TCP	Telnet	RFC 854
25	TCP	Simple Mail Transfer Protocol (SMTP)	RFC 5321
53	TCP and UDP	Domain Name Server (DNS)	RFC 1034-1035
67, 68	UDP	Dynamic Host Configuration Protocol (DHCP)	RFC 2131
69	UDP	Trivial File Transfer Protocol (TFTP)	RFC 1350
80	TCP	HyperText Transfer Protocol (HTTP)	RFC 2616
110	TCP	Post Office Protocol (POP3)	RFC 1939
119	TCP	Network News Transport Protocol (NNTP)	RFC 8977
123	UDP	Network Time Protocol (NTP)	RFC 5905
135-139	TCP and UDP	NetBIOS	RFC 1001-1002
143	TCP and UDP	Internet Message Access Protocol (IMAP4)	RFC 3501
161, 162	TCP and UDP	Simple Network Management Protocol (SNMP)	RFC 1901-1908, 3411-3418
179	TCP	Border Gateway Protocol (BGP)	RFC 4271
389	TCP and UDP	Lightweight Directory Access Protocol	RFC 4510
443	TCP and UDP	HTTP with Secure Sockets Layer (SSL)	RFC 2818
500	UDP	Internet Security Association and Key Management Protocol (ISAKMP) / Internet Key Exchange (IKE)	RFC 2408 - 2409
636	TCP and UDP	Lightweight Directory Access Protocol over TLS/SSL (LDAPS)	RFC 4513
989/990	TCP	FTP over TLS/SSL	RFC 4217

Protocollo TCP: chiusura normale

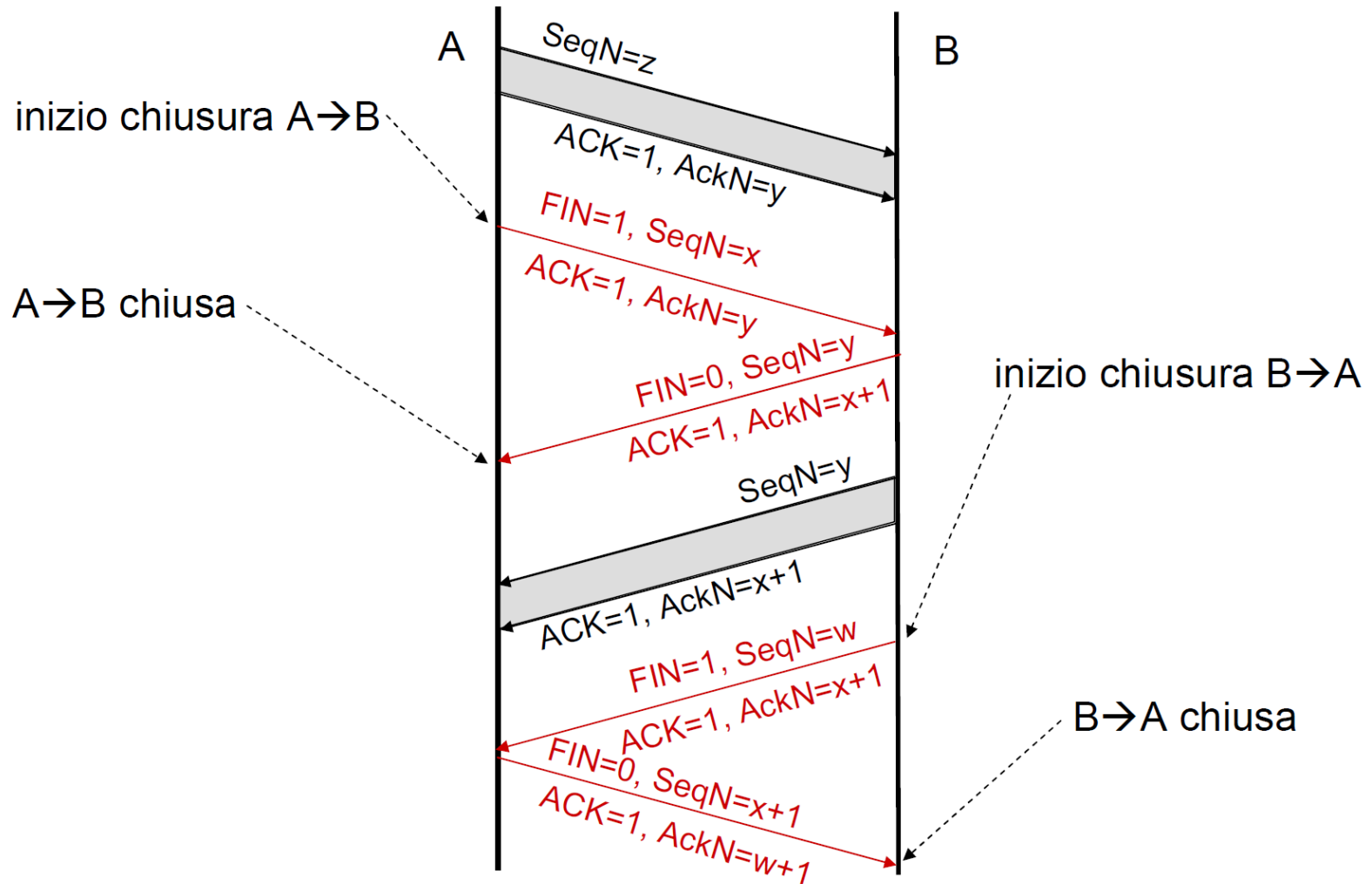


TCP sceglie di realizzare la chiusura in modalità simplex

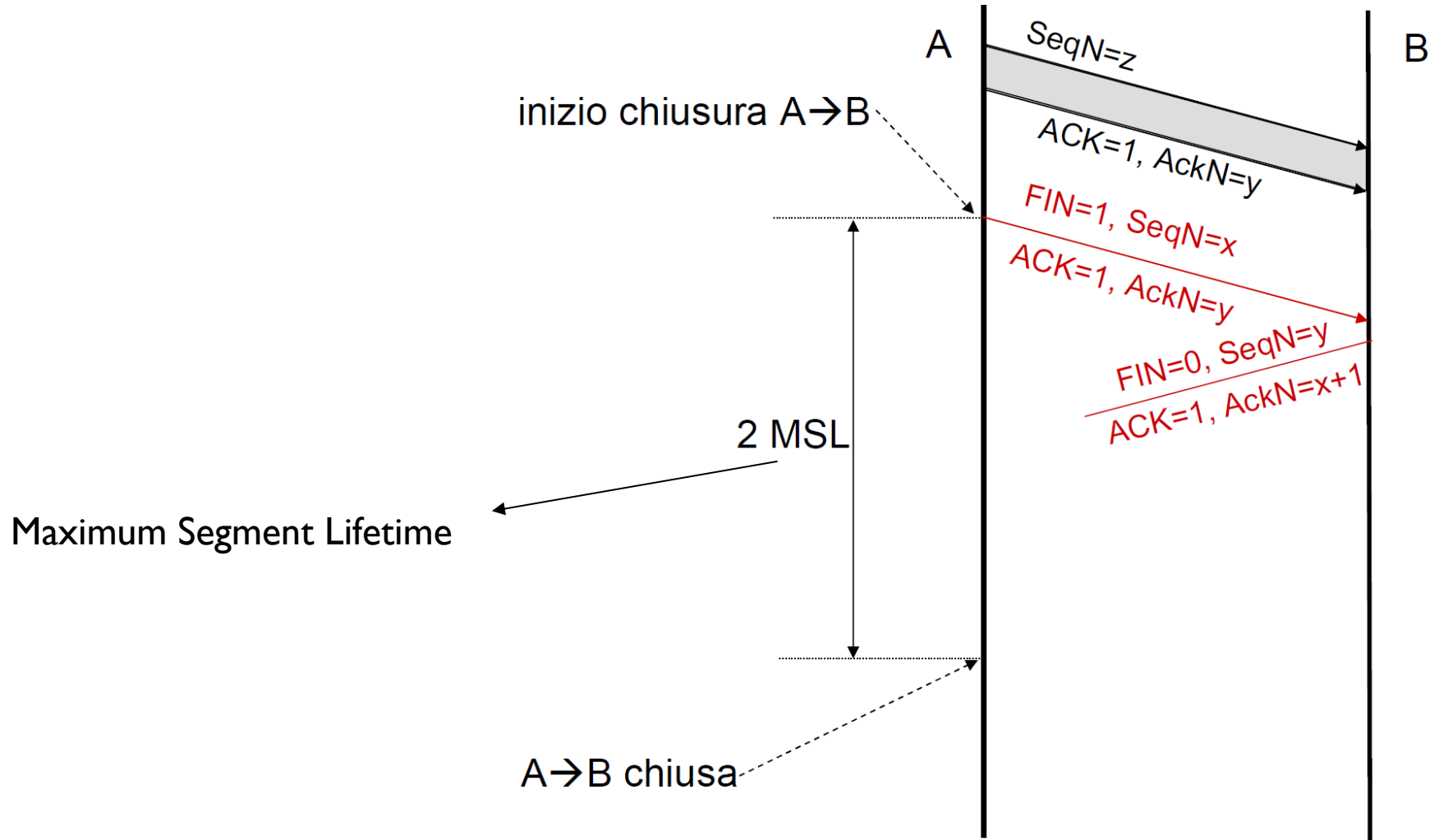
- le due direzioni vengono rilasciate in modo indipendente
- il TCP che intende terminare la trasmissione emette un segmento con $FIN=1$
- ▶ quando questo segmento riceve l'ACK la direzione si considera chiusa
- ▶ se dopo un certo tempo non arriva l'ACK il mittente del FIN rilascia comunque la connessione
 - l'altra direzione può continuare a trasmettere dati finché non decide di chiudere



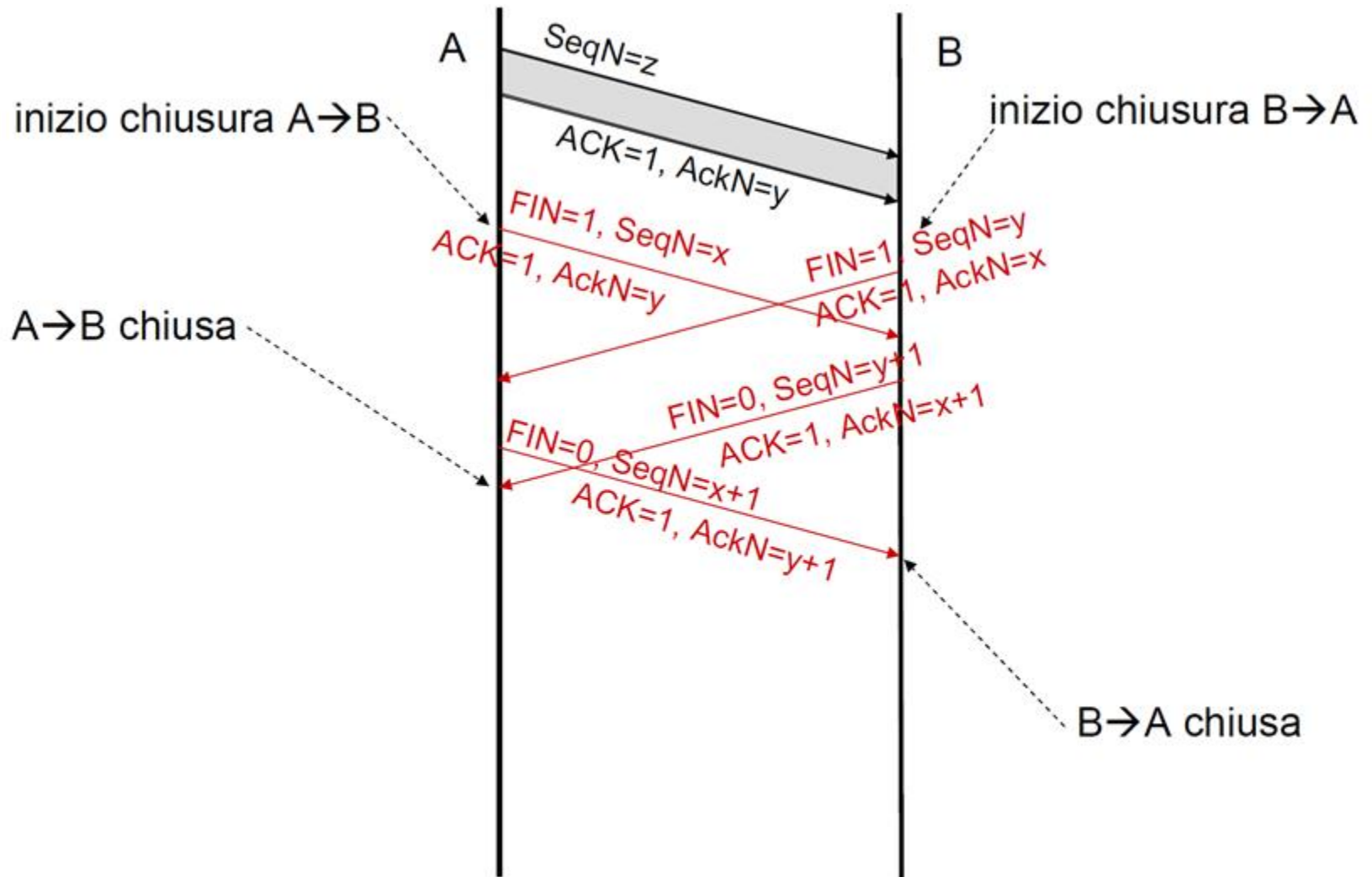
Protocollo TCP: chiusura normale



Protocollo TCP: chiusura con Ack perso



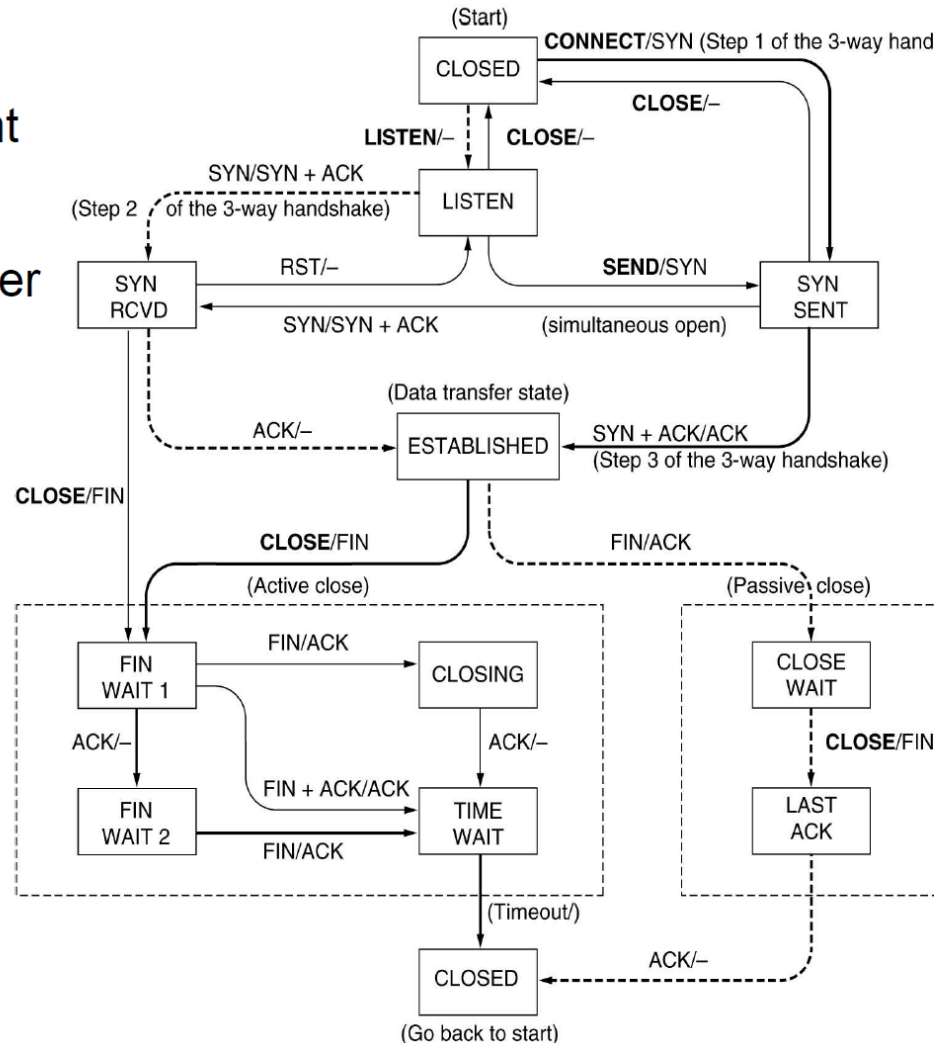
Protocollo TCP: chiusura contemporanea



La macchina a stati finiti del TCP



- Linee marcate
 - azioni tipiche di un client
- Linee tratteggiate
 - azioni tipiche di un server
- Linee leggere
 - eventi inusuali
- Transizioni
 - causa/effetto



Da A.S. Tanenbaum, "Reti di Calcolatori"

La macchina a stati finiti del TCP



State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for Ack
SYN SENT	The client has started to open a connection
ESTABLISHED	Normal data transfer state
FIN WAIT 1	Client has said it is finished
FIN WAIT 2	Server has agreed to release
TIMED WAIT	Wait for pending packets ("2MSL wait state")
CLOSING	Both Sides have tried to close simultaneously
CLOSE WAIT	Server has initiated a release
LAST ACK	Wait for pending packets

2MSL = due volte il tempo massimo di vita di un segmento (30 – 120 sec)



Richiamo: IP Checksum



Il valore del **checksum** si calcola:

- trattando l'intestazione come una sequenza di interi a 16 bit (2 byte alla volta) dove checksum vale 0;
- sommando gli interi (con aritmetica in complemento a 1);
- prendendo il complemento a 1 (cioè il not binario) del risultato.

Essendo il checksum calcolato da chi ha trasmesso il frame, già compreso nel frame stesso, il checksum calcolato dal ricevitore (compreso il checksum ricevuto) dovrà avere tutti i bit a 1

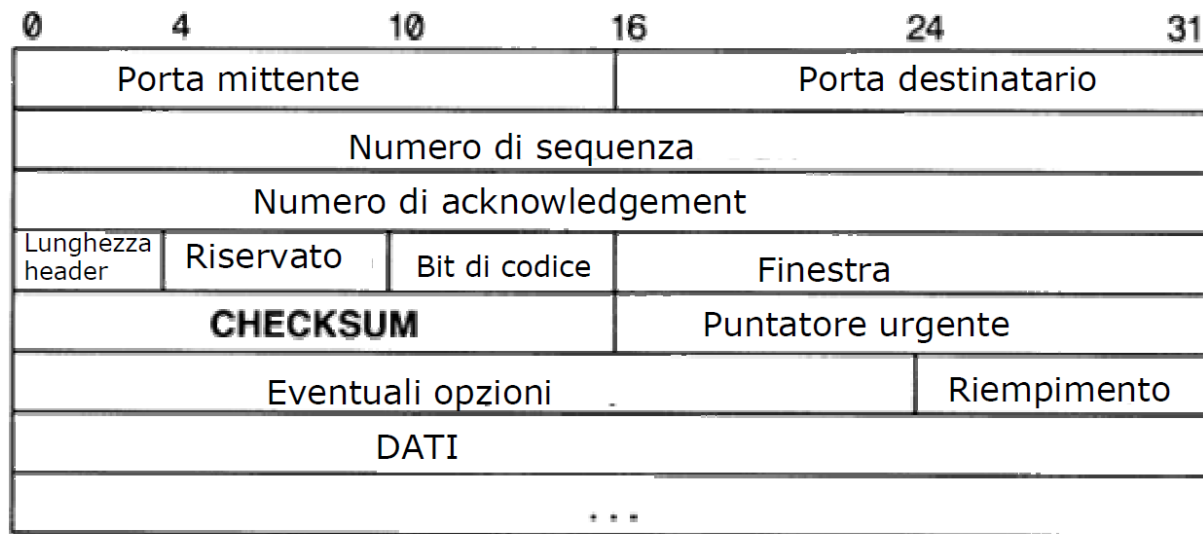
Versione (4)	IHL(4)	Tipo di Servizio(8)	Lunghezza Totale(16)	
Id del Datagramma(16)			Flag(3)	Offset di Frammentazione(13)
Time To Live(8)	Protocollo(8)	Checksum dello header(16)		
Indirizzo IP sorgente(32)				
Indirizzo IP destinazione(32)				
Opzioni			Padding	
Dati				

TCP Checksum



TCP utilizza un meccanismo analogo, inoltre:

- aggiunge una **pseudo-intestazione** che contiene, fra l'altro, l'IP della sorgente e della destinazione;
- prende in considerazione anche i dati.



Differenze tra TCP e UDP Checksum



- ▶ Per TCP il calcolo del checksum è **obbligatorio**
- ▶ Il checksum è un campo di controllo *end-to-end*: è calcolato dal mittente e verificato dal ricevitore del pacchetto.
- ▶ Se il pacchetto UDP è errato, viene scartato silenziosamente, senza generazione di messaggi d'errore.
- ▶ Se il pacchetto TCP è errato, deve essere inviato nuovamente.
- ▶ **L'opzionalità dei checksum di UDP** implica una velocità maggiore del trasferimento dei dati quando disabilitata (con tutti i rischi correlati)



Approfondimento TCP: tecnica piggybacking

- ▶ Quando arriva un frame di dati, la destinazione non invia subito un **acknowledgement separato** di conferma ma aspetta che lo strato network gli passi il successivo pacchetto
- ▶ L' *acknowledgement* viene aggiunto al frame di dati in uscita usando il campo ack nell'header del segmento TCP
- ▶ In questo modo l'*acknowledgement* si procura un passaggio "gratis" insieme al successivo frame dati trasmesso.
- ▶ La tecnica che consiste nel ritardare gli acknowledgement in uscita in modo da agganciarli al successivo frame di dati è chiamata **piggybacking** (letteralmente "trasportare in groppa").



Approfondimento TCP: controllo del flusso

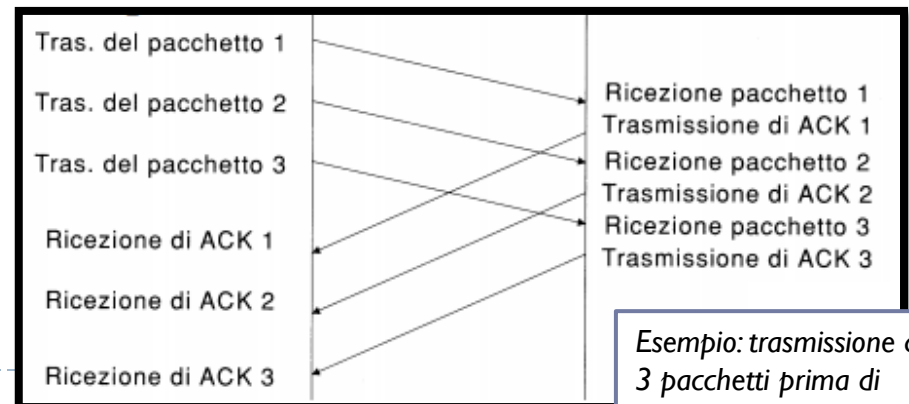
- ▶ Il problema del controllo del flusso in TCP è affrontato, per alcuni aspetti, in modo simile a quanto previsto nel livello data link
- ▶ Per esempio, entrambi gli approcci, si basano sull'uso della **sliding window** al fine di impedire che un trasmettitore troppo veloce possa sommergere di dati un ricevitore lento
- ▶ La gestione del buffering in TCP è dinamica: si utilizza una finestra di dimensione variabile aggiornata durante la trasmissione

Porta Sorgente(16)			Porta destinazione(16)		
Numero di Sequenza(32)					
Numero di Acknowledgement(32)					
HLEN(4)	Riservati(6)	Flag(6)	Window(16)		
Checksum(16)			Urgent Pointer(16)		
Opzioni				Padding	
Dati					

Per indicare la dimensione del buffer si usa il campo *window* nell'header TCP

Approfondimento TCP: controllo del flusso ★

- ▶ La tecnica della finestra scorrevole (**sliding windows**) permette al trasmettitore di poter continuare ad inviare un certo numero N (dimensione della finestra) di pacchetti successivi all'ultimo per cui ha ricevuto l' *acknowledgement*, in altre parole permette di trasmettere fino ad altri N pacchetti mentre si è in attesa di ricevere il riscontro di un pacchetto precedentemente inviato. In tal modo la rete viene utilizzata anche nei periodi di attesa
- ▶ In TCP, diversamente da altri protocolli, la dimensione massima della finestra di trasmissione **è variabile** ed è **determinata anche dal ricevente**
- ▶ Nella maggior parte degli altri protocolli basati su *sliding window* la dimensione massima della finestra è costante



Approfondimento TCP: controllo del flusso

- ▶ Per regolare il flusso occorre un meccanismo che tenga conto di due aspetti:
 1. **Capacità della rete:** assenza di timeout dovuti alla congestione (tipicamente è la causa principale nelle reti wired) o errori di trasmissione (tipicamente è la causa principale nelle reti wireless)
 2. **Capacità del buffer del ricevente:** quanti byte il ricevente è in grado di ricevere
- ▶ Quando viene stabilita una connessione si deve scegliere una dimensione adatta per la finestra: non è sufficiente l'analisi della finestra di ricezione del ricevente (regolata tramite il campo *window* dell'*header* TCP) ma occorre tener conto anche della capacità della rete



Approfondimento TCP: controllo del flusso

Primo aspetto: capacità della rete

- ▶ Quando il carico applicato a una rete è superiore a quello che può gestire, si verifica una **congestione**
- ▶ La soluzione alle congestioni è la diminuzione della velocità dei dati
- ▶ Per rilevare uno stato di congestione basta osservare che nelle reti wired la maggior parte dei timeout è causato da congestione e non da errori di trasmissione
- ▶ E' necessario che il mittente imposti una **finestra di congestione**:

il massimo numero di byte che la rete è in grado di trasmettere senza che si verifichino dei timeout che segnalino la perdita di dati



Approfondimento TCP: controllo del flusso

Secondo aspetto: capacità del buffer del ricevente

- ▶ Il ricevente può specificare una finestra in base alla dimensione del suo buffer di ricezione: **finestra di ricezione**
- ▶ Se il mittente si attiene a questa dimensione, non si verificheranno problemi dovuti al traboccamento del buffer all'estremità ricevente, (potrebbero ancora verificarsi a causa della congestione interna alla rete)



Approfondimento TCP: controllo del flusso ★

Secondo aspetto: capacità del buffer del ricevente

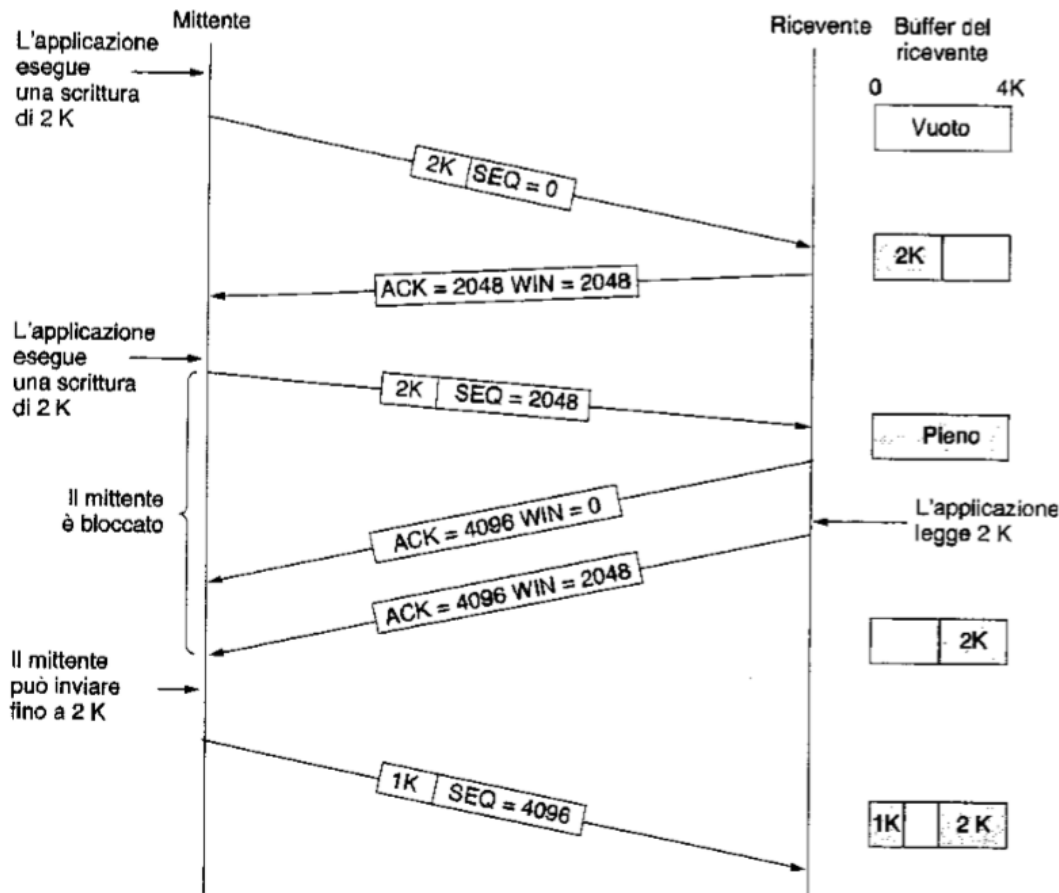


Figura 6.34. La gestione della finestra (window) in TCP.

Quando la finestra è ZERO, il mittente non può di norma inviare segmenti, con due eccezioni:

- 1) è possibile inviare dati urgenti
- 2) il mittente può inviare un segmento di 1 byte, per fare in modo che il ricevente annunci di nuovo il successivo byte previsto e la dimensione della finestra.

Lo standard TCP fornisce esplicitamente questa possibilità per scongiurare un blocco in caso di perdita di un annuncio della finestra.

Approfondimento TCP: controllo del flusso ★

Analisi del problema: **analogia idraulica**

- ▶ Nella Figura 6.36(a) una conduttura di grande diametro porta a un ricevente di piccola capacità. Finché il mittente non invia più acqua di quella che può essere contenuta nel catino, non ci saranno perdite.
- ▶ Nella Figura 6.36(b), il fattore limitante non è la capacità del catino ma la capacità di trasporto della rete. Se giunge troppa acqua rapidamente, parte di essa verrà persa (in questo caso per traboccamento del tubo).

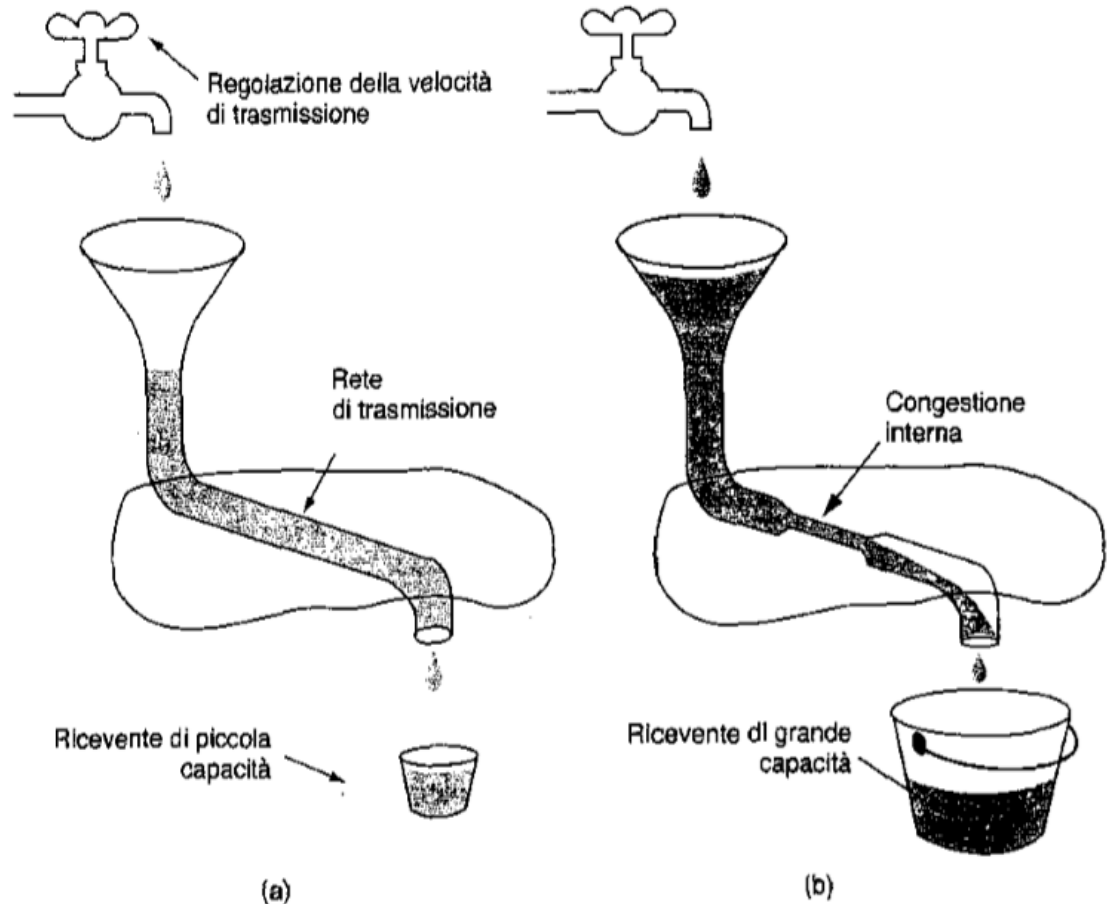


Figura 6.36. (a) Una rete veloce che alimenta un ricevente a bassa capacità.
(b) Una rete lenta che alimenta un ricevente ad alta capacità.

Approfondimento TCP: controllo del flusso

Gestione del flusso (I/4)

- ▶ Ogni mittente mantiene dunque due finestre: la **finestra di congestione** e la **finestra di ricezione**.
- ▶ Ogni finestra riflette il *numero di byte* che il mittente può trasmettere prima di ricevere una conferma.
- ▶ Il numero di byte che possono essere inviati corrisponde alla più piccola delle due finestre:

$\text{finestra massima} = \min(\text{finestra di congestione}, \text{finestra di ricezione})$



Approfondimento TCP: controllo del flusso

Gestione del flusso (2/4)

- ▶ Quando viene stabilita una connessione, il mittente **inizializza la finestra di congestione alla massima quantità di byte che la rete può trasferire** (segmento massimo trasferibile) e invia un segmento di questa dimensione
 - ▶ La dimensione del segmento massimo trasferibile sulla connessione: MSS – Maximum Segment Size, un valore che viene stimato tenendo in considerazione anche il valore della MTU (Maximum Transfer Unit)
 - ▶ Se questo segmento riceve un *ack* prima della scadenza del timer, aggiunge al valore della finestra di congestione un numero pari alla dimensione del segmento, in modo che la sua dimensione sia il doppio della dimensione massima del segmento, quindi invia due segmenti.
-



Approfondimento TCP: controllo del flusso

Gestione del flusso (3/4)

- ▶ Quando ciascuno di questi segmenti riceve un acknowledgement, la dimensione della finestra di congestione viene aumentata di un valore pari alla dimensione massima del segmento
- ▶ In pratica ogni gruppo di segmenti inviati che riceve un *ack* raddoppia la dimensione della finestra di congestione
- ▶ La finestra di congestione continua a **crescere in modo esponenziale** fino a un timeout o al raggiungimento del valore della finestra del ricevente (indicato nel campo window dell'header TCP)
- ▶ Questo è l'algoritmo di Jacobson ed è chiamato avvio lento (**slow start**), pur non essendo per nulla lento: è esponenziale. **Tutte le implementazioni TCP devono supportarlo.**



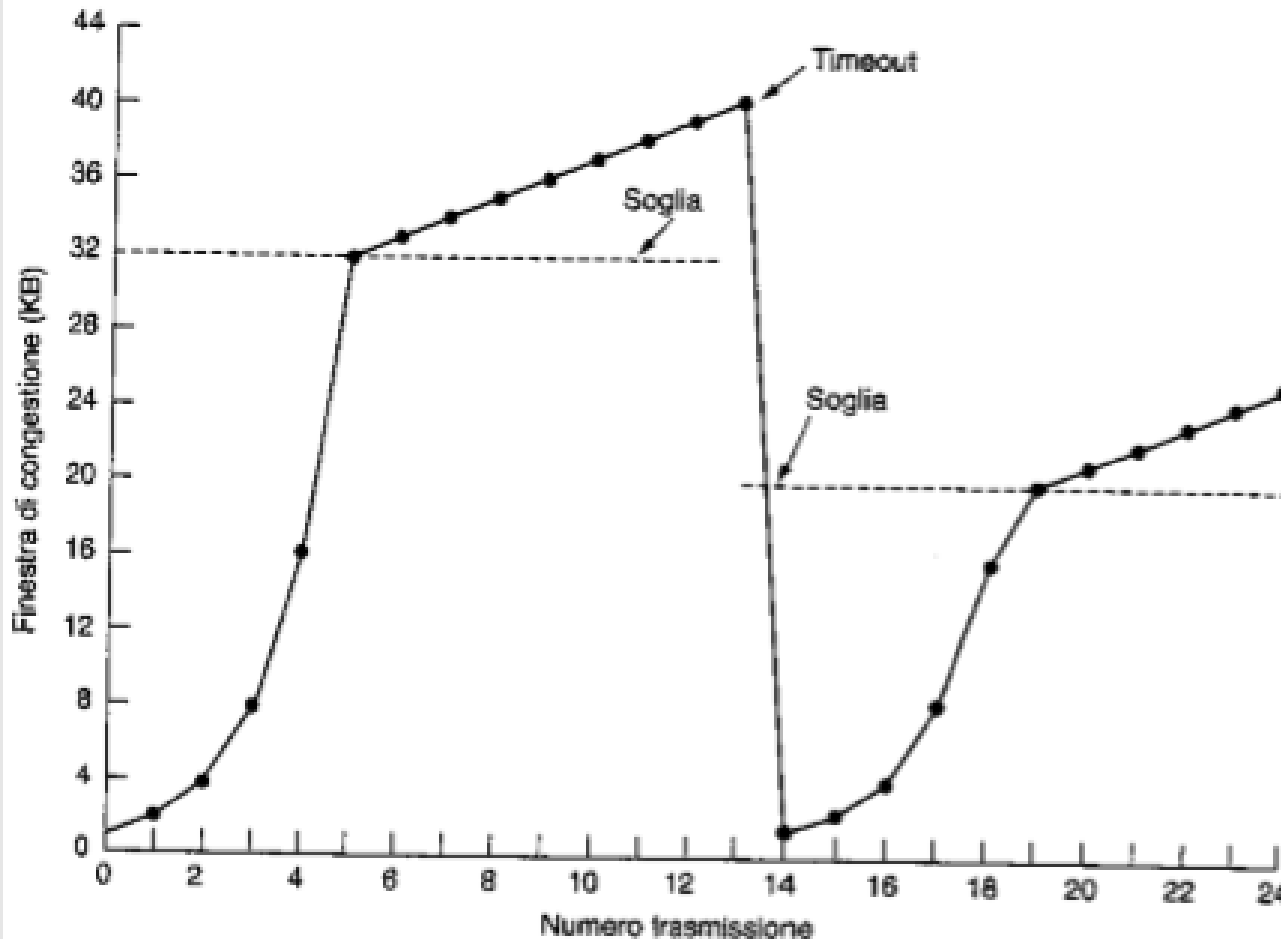
Approfondimento TCP: controllo del flusso

Gestione del flusso (4/4)

- ▶ L'algoritmo di controllo della congestione utilizza anche un valore soglia (**threshold**) inizialmente impostato al valore 64 KB
- ▶ Al verificarsi di un *timeout*, la soglia viene impostata alla metà della finestra di congestione corrente e la finestra di congestione viene reimpostata alla dimensione massima di un segmento
- ▶ L'avvio lento viene utilizzato per determinare la capacità della rete ma la crescita esponenziale termina al raggiungimento del valore soglia; da quel punto in poi le trasmissioni avvenute con successo aumentano la finestra di congestione in modo lineare (algoritmo **congestion avoidance**): si aggiunge la dimensione di un MSS (il valore impostato inizialmente)
- ▶ In caso di timeout l'algoritmo suppone che sia probabilmente accettabile tagliare a metà la finestra di congestione e poi lavora gradualmente partendo da questo valore



Approfondimento TCP: controllo del flusso ★



Esempio: algoritmo di gestione della congestione

Approfondimento TCP: impostazione dei timer



Impostazione del *timeout* di ritrasmissione

- ▶ TCP utilizza più timer (almeno dal punto di vista teorico), il più importante dei quali è il **timer di ritrasmissione**.
- ▶ Alla ricezione di un segmento viene avviato un timer di ritrasmissione: se il segmento riceve un ack prima della scadenza il timer viene fermato; altrimenti il segmento viene ritrasmesso e il timer riavviato
- ▶ Il problema del miglior valore a cui impostare il timer è cruciale, se il timeout impostato:
 - ▶ è **troppo breve** si verificano ritrasmissioni non necessarie (peggiora la congestione della rete)
 - ▶ è **troppo lungo** le prestazioni ne risentiranno a causa del lungo ritardo di ritrasmissione dei pacchetti persi
- ▶ Viene utilizzato un algoritmo altamente dinamico che regola costantemente il timeout in base alle prestazioni della rete che devono essere costantemente monitorate.



Approfondimento TCP: reti wireless



TCP su rete wireless

- ▶ I protocolli di livello trasporto **dovrebbero essere totalmente indipendenti dalla tecnologia utilizzata ai livelli sottostanti dello stack TCP/IP**: l'esecuzione di IP su rete *wired* o *wireless* non dovrebbe modificarne il comportamento
- ▶ Nella pratica, però, la maggior parte delle implementazioni TCP è stata ottimizzata sulla base di assunzioni che valgono nelle reti *wired* ma non nelle reti *wireless*
- ▶ In particolare, si è visto come **l'algoritmo di controllo della congestione assuma che i timeout siano provocati da congestione nei router e non da errori di trasmissione: ma nelle reti wireless accade l'esatto contrario!**
- ▶ Conseguentemente la partenza rallentata (approccio *slow start*) in seguito a un timeout, provoca sulle reti wireless un degrado prestazionale, in quanto sarebbe più efficiente ritrasmettere i dati persi in modo rapido invece di un approccio conservativo (con l'obiettivo di ridurre la congestione)
- ▶ Diversi approcci puntano a **modificare il comportamento degli algoritmi utilizzati in TCP in base alle tecnologie dei livelli sottostanti**

